

How do different types of optimization algorithms affect the performance of machine learning models?

Mathematics: analysis and approaches HL | Internal assessment

1 | Introduction

Machine learning (ML) is a branch of artificial intelligence that uses statistical techniques to give computer systems the ability to “learn” from data, without being explicitly programmed. Machine learning algorithms build a mathematical model based on sample data, known as **training data**, to make predictions or decisions without being explicitly programmed to perform the task.

Today, machine learning models are used in our everyday lives, from personalized recommendations on Netflix to facial recognition on our phones. At the core of the training process are **optimization algorithms**, which are used to teach the model to more accurately predict an output given an input. There are many methods of optimizing this loss function; this exploration will compare two common methods: **gradient descent** and **simulated annealing**. As an aspiring computer science researcher interested in studying natural language processing, having a strong understanding of these fundamental ML concepts will be invaluable.

2 | The regression task

The specific ML problem chosen for this exploration will be a regression task. **Regression** is a **supervised learning**¹ algorithm that models the relationship between a scalar dependent variable y and an independent variables x .

To look at the effect of each optimization algorithm on the performance of a regression model, we will apply both on a real-world dataset of information on cars. For instance, let us define our problem to be investigating the relationship between the horsepower rating (in hp) of a car and its fuel efficiency (in mpg).

A performant regression model for the problem would help us accurately predict the fuel efficiency of a car given its horsepower, both interpolating and extrapolating. This can be measured statistically, with the **coefficient of determination** R^2 , as well as in comparison to the analytical solution (see Section 6).

¹**Supervised learning** is a category of machine learning tasks where a model learns a function that maps an input to an output based on example input-output pairs (from the **training data**).

Cleaning and extracting the relevant data from the dataset, we get the following scatterplot:

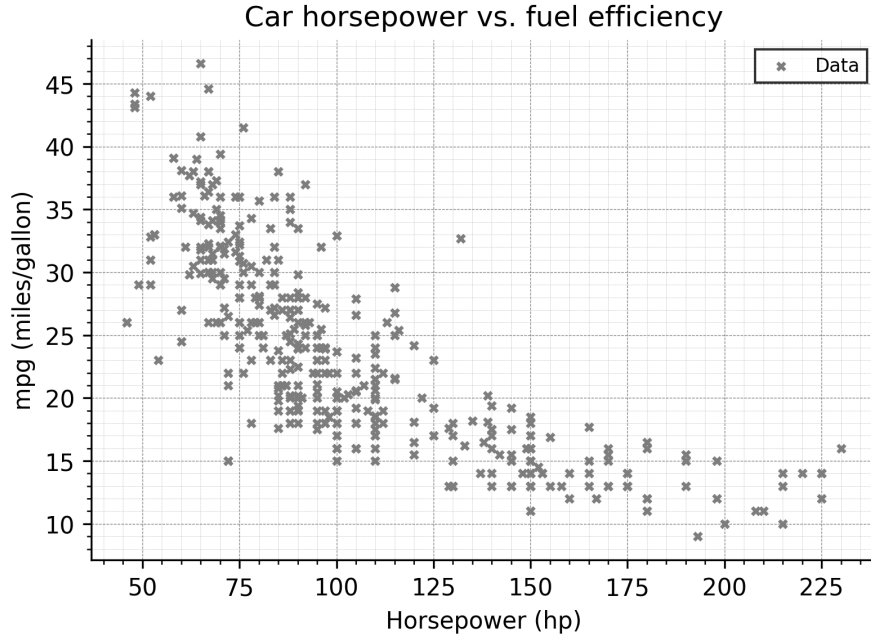


Figure 1: Scatterplot of car data. Derived from dataset published by Elmetwally (2023).

From the scatterplot, we observe a negative exponential relationship between the two variables, and hence we will attempt to optimize an exponential regression model to fit the data.

3 | Optimization and loss functions

Optimization is the process of finding the best solution to a problem, which in this case is finding the equation of the line that best fits the data—according to some measure. In ML, this measure is called a loss function; the goal of optimization is to minimize the loss function.

There are many types of loss functions, which depend on the problem and type of data. This includes “square of Euclidean distance, cross-entropy, contrast loss, hinge loss, information gain,” etc (Sun et al., 2019).

A common loss function is the sum of squared residuals, defined as

$$L = \sum_{i=1}^n (y_i - f(x_i))^2 \quad (1)$$

where y_i is the i^{th} actual value of the variable we are trying to predict (“ground truth”), $f(x_i)$ is the predicted value of y_i given the i^{th} value of the independent variable x , and n is the number of data points. In this case, y is the fuel efficiency of the car, x is the horsepower rating of the car, and $f(x)$ is the predicted fuel efficiency of the car given its horsepower rating.

4 | Gradient descent and stochastic gradient descent (SGD)

Gradient descent is an optimization algorithm that minimizes a function by taking steps proportional to the negative of the gradient of the function at the current point.

4.1 | The gradient descent algorithm

Algorithm Step

Since this is an exponential regression task,

the function $f(x)$ is a line with equation

$$f(x) = a \cdot e^{b(x-h)} + k.$$

Each squared residual comes from the difference between the actual value of y and the predicted value of y , $f(x_i)$. Therefore, the loss function can be rewritten as

$$L = \sum_{i=1}^n (y_i - a \cdot e^{b(x-h)} - k)^2 \quad (2)$$

Calculating the summation, we can substitute y_i and x_i with each datapoint's value, leaving the loss function in terms of the parameters a , b , h , k as the targets for optimization.

Application Example

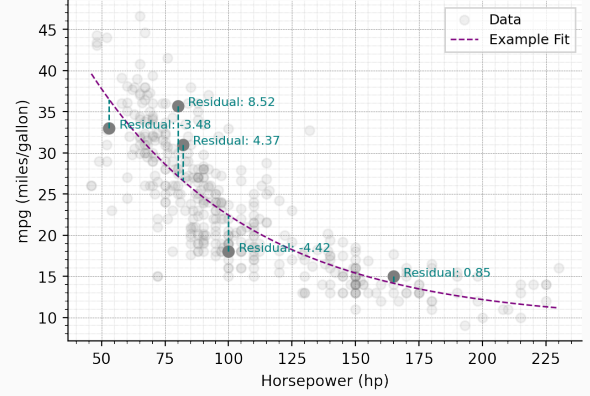


Figure 2: As an example, residuals between datapoints and an example regression line

Initialization

We first define an initial solution (a **guess**), where we choose arbitrary but sensible values for the parameters

- a (scaling factor),
- b (growth factor),
- h (horizontal shift), and
- k (vertical shift).

Let's call these a_0 , b_0 , h_0 , and k_0 .

Let's choose:

- $a_0 = 1$
- $b_0 = -0.05$, as we recognize an exponential decay shape.
- $h_0 = 80$, as that brings the y-intercept around 50 mpg.
- $k_0 = 10$, as the horizontal asymptote seems to exist around 10 mpg.

Computing the gradient

Taking the partial derivative of the loss function with respect to each individual parameter gives us the instantaneous rate of change of each parameter, which is the slope of the loss function at that point. Together, they make up the **gradient** of the loss function, which is essentially a vector that points in the direction of the steepest ascent of the loss function. The idea is that, by taking steps in the opposite direction of the gradient, we will eventually reach the minimum of the loss function.

→ Derivative with respect to a

From Equation 2, we have

$$L(a, b, h, k) = \sum (y_i - a \cdot e^{b(x_i - h)} - k)^2$$

By the chain rule,

$$\begin{aligned} \frac{\partial L}{\partial a} &= 2 \times (y_i - a \cdot e^{b(x_i - h)} - k) \times -e^{b(x_i - h)} \\ \frac{\partial L}{\partial a} &= -2 \times \sum e^{b(x_i - h)} (y_i - a \cdot e^{b(x_i - h)} - k) \end{aligned} \tag{3}$$

→ **Derivative with respect to b**

By the chain rule,

$$\begin{aligned}\frac{\partial L}{\partial b} &= 2 \times (y_i - a \cdot e^{b(x_i-h)} - k) \times a \cdot e^{b(x_i-h)} \cdot (x_i - h) \\ \frac{\partial L}{\partial b} &= -2 \times \sum a \cdot e^{b(x_i-h)} (x_i - h) (y_i - a \cdot e^{b(x_i-h)} - k)\end{aligned}\tag{4}$$

→ **Derivative with respect to h**

By the chain rule,

$$\begin{aligned}\frac{\partial L}{\partial h} &= 2 \times (y_i - a \cdot e^{b(x_i-h)} - k) \times -ab \cdot e^{b(x_i-h)} \\ \frac{\partial L}{\partial h} &= 2 \times \sum ab \cdot e^{b(x_i-h)} (y_i - a \cdot e^{b(x_i-h)} - k)\end{aligned}\tag{5}$$

→ **Derivative with respect to k**

By the chain rule,

$$\begin{aligned}\frac{\partial L}{\partial k} &= 2 \times (y_i - a \cdot e^{b(x_i-h)} - k) \times -1 \\ \frac{\partial L}{\partial k} &= -2 \times \sum (y_i - a \cdot e^{b(x_i-h)} - k)\end{aligned}\tag{6}$$

Altogether, the gradient is

$$\begin{bmatrix} \frac{\partial L}{\partial a} \\ \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial h} \\ \frac{\partial L}{\partial k} \end{bmatrix}\tag{7}$$

Take our initial parameters. Substituting them into the gradient, we get the following values:

$$\begin{bmatrix} 15921.6831 \\ 87049814.8282 \\ 796.0842 \\ 17774.9488 \end{bmatrix}\tag{8}$$

Parameter update

Each component of the gradient is substituted into the formula for the step size, which is the amount by which each parameter is changed per iteration. The step size is multiplied by a learning rate γ , which is a constant (a **hyperparameter**) that controls the rate of parameter change.

- Step size_a = $\frac{\partial L}{\partial a} \times \gamma$
- Step size_b = $\frac{\partial L}{\partial b} \times \gamma$
- Step size_h = $\frac{\partial L}{\partial h} \times \gamma$
- Step size_k = $\frac{\partial L}{\partial k} \times \gamma$

Let's set the learning rate γ to 10^{-9} initially.

- Step size_a = $15921.6831 \times 10^{-9}$
- Step size_b = $87049814.8282 \times 10^{-9}$
- Step size_h = 796.0842×10^{-9}
- Step size_k = $17774.9488 \times 10^{-9}$

This learning rate γ must be fine-tuned because a small learning rate might lead to slow convergence, while a large learning rate can cause overshooting and instability.

The step sizes are then subtracted from the original parameters (the parameters from the previous iteration or the initial values) to get new values.

$$\begin{aligned} \text{New value of a} &= 1 - 15921.6831 \times 10^{-9} \\ &= 0.999984078317 \end{aligned}$$

$$\begin{aligned} \text{New value of b} &= -0.05 - 87049814.8282 \times 10^{-9} \\ &= -0.13704981428 \end{aligned}$$

$$\begin{aligned} \text{New value of h} &= 80 - 796.0842 \times 10^{-9} \\ &= 79.9999992039 \end{aligned}$$

$$\begin{aligned} \text{New value of k} &= 10 - 17774.9488 \times 10^{-9} \\ &= 9.99998222505 \end{aligned}$$

Convergence check and iteration

This process of substituting the parameter values into the gradient, then calculating the new step sizes, and finally the new parameters, is repeated until the loss function converges to a minimum.

Convergence can be determined by setting a threshold for the step size (when it is sufficiently small) or by limiting the number of iterations.

4.2 | Gradient descent on our problem

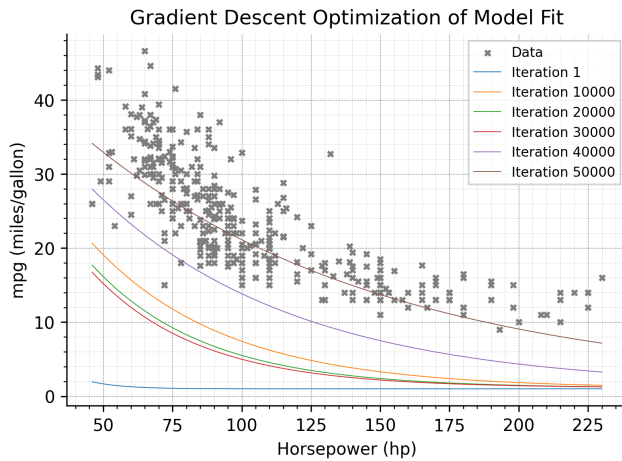


Figure 3: Gradient descent on our problem

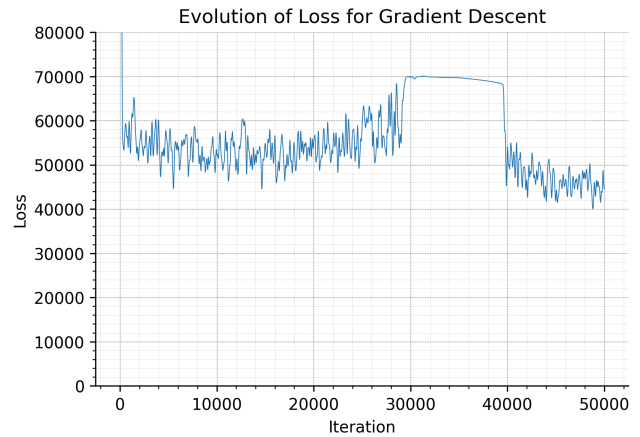


Figure 4: Loss function over iterations

The algorithm ran through 50,000 iterations.

While the difference between each iteration is minimal, these changes lead to the parameters slowly converging over many iterations on a satisfactory solution (brown line, Iteration 50000) that appears to fit the scatterplot. This is reflected in the loss graph, which decreases over time although there is significant noise.

5 | Simulated annealing (SA)

Simulated annealing is a probabilistic technique inspired by metallurgy that uses the concept of ‘temperature’ to control the search process.

5.1 | The simulated annealing algorithm

Initialization

We define an initial solution, where we choose values for the parameters.

We also set an initially high **temperature** T_0 . The idea is that, at high temperatures, the algorithm is more likely to accept changes that increase the function value (or ‘energy’), allowing it to explore a wide range of solutions. As the temperature decreases, the algorithm becomes more selective, focusing on refining and improving the current solution. This process helps SA to potentially escape local minima and converge towards a global minimum.

Let’s choose the same initial solutions as we did in gradient descent:

- $a_0 = 1$
- $b_0 = -0.05$
- $h_0 = 80$
- $k_0 = 10$

We set a sufficiently high temperature of $T_0 = 500$.

Iteration

A new candidate solution is generated by perturbing the current solution. This is done by adding a random value to each parameter.

Using `new_params = params + np.random.normal(0, 1, size=len(params))` in the program implementation, we get the following perturbed values:

- $a = 1.54626094$
- $b = -0.08207048$
- $h = -80.84881705$
- $k = -10.03802701$

Evolution and acceptance

The new candidate solution is evaluated by the difference in the loss function, ΔL .

If the candidate solution is better than the current solution, that is, $\Delta L > 0$, then the candidate solution is accepted as the new current solution.

If the candidate solution is worse than the current solution, that is, $\Delta L < 0$, then the candidate solution is accepted as the new current solution with a probability of

$$P(\text{accept}) = e^{-\frac{\Delta L}{T}} \quad (9)$$

where T is the current temperature.

In this iteration, the candidate solution is better than the current solution, so we accept the candidate solution as the new current solution:

$$\Delta L = 57943.6433449937$$

Cooling

At the end of each iteration, the temperature is decreased by a cooling factor α . This is done to reduce the probability of accepting worse solutions as the algorithm progresses.

For instance, a simple exponential decay might be

$$T_{\text{new}} = \alpha \times T_{\text{old}} \quad (10)$$

where α is a constant < 1 , controlling the rate of temperature decrease.

Let's choose $\alpha = 0.98$, to give the algorithm more time to explore other minima.

The new temperature is then

$$\begin{aligned} T_{\text{new}} &= 0.98 \times T_{\text{old}} \\ &= 0.98 \times 500 \\ &= 490 \end{aligned} \quad (11)$$

Termination

The algorithm repeats the iteration step until a stopping criterion is met. This could be a sufficiently low temperature, a maximum number of iterations, or a minimal change in the loss function over a certain number of iterations.

5.2 | Simulated annealing on our problem

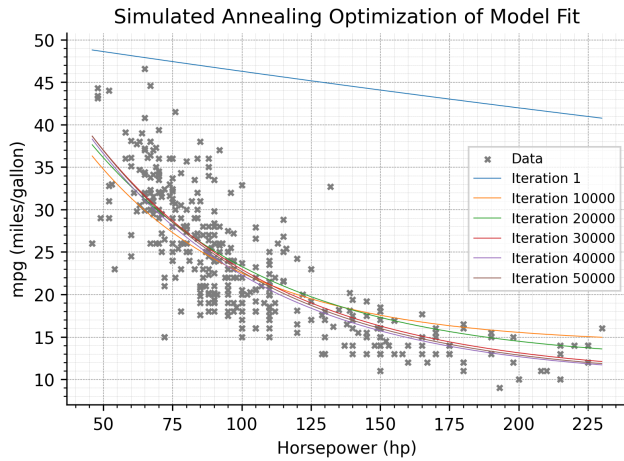


Figure 5: Simulated annealing on our problem

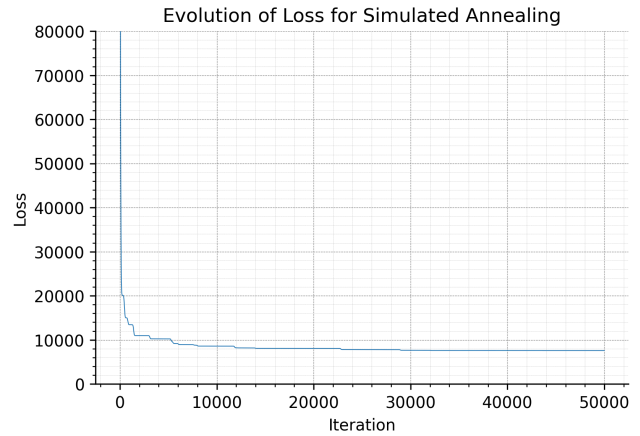


Figure 6: Loss function over iterations

The algorithm ran through 50,000 iterations.

The loss graph showed a steep drop, indicating that simulated annealing was quickly able to find a satisfactory solution within 10,000 iterations before converging on a minimum (appears as a horizontal asymptote). The final solution (brown line, Iteration 50000) appears to fit the scatterplot well.

6 | Analysis

To evaluate the performance of the two optimization algorithms, we will compare their results to an analytical solution calculated using the Levenberg-Marquardt algorithm, a method for solving non-linear least squares problems. This algorithm resulted in a theoretical best-fit line for the data which is shown in green in Figure 7.

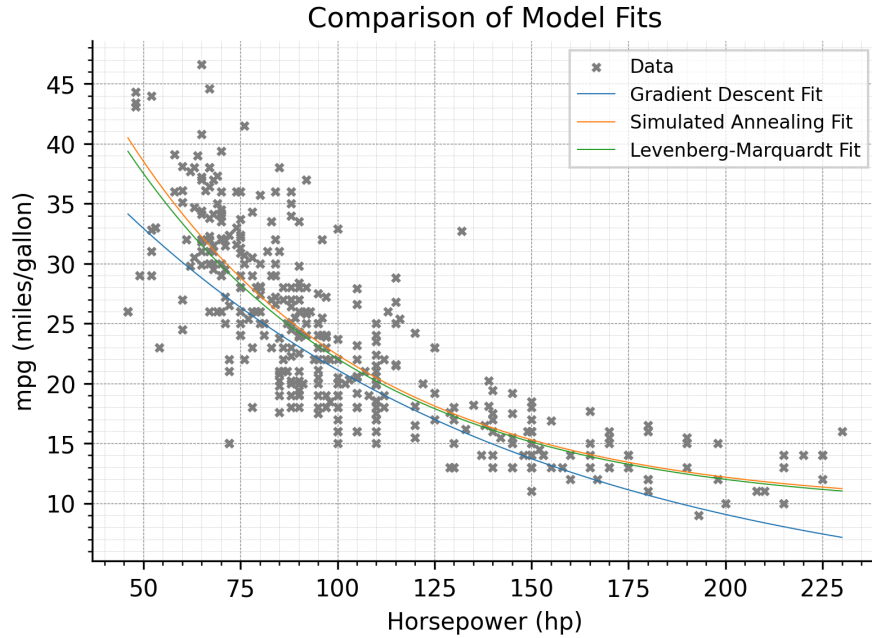


Figure 7: Comparison between two optimizers and the analytical line of best fit

Numerically, the final iteration of each algorithm resulted in the following values:

Algorithm	a	b	h	k	Loss L
Gradient descent	50.010563	-0.009311	1.038900	1.226458	70446.186542
Simulated annealing	54.825537	-0.015162	2.979578	10.094089	7579.026930
Analytical solution (Levenberg-Marquardt)	41.927	-0.016	24.890	9.453	7514.335123

Table 1: Final values for the parameters of the equation, and the loss function L

Comparing the solution found by the two optimization algorithms to the analytical solution suggests that in this problem, simulated annealing performed significantly better than gradient descent.

Simulated annealing was able to find a solution that was closer to the analytical solution (the theoretical best solution), with a difference in the loss value to the analytical solution of 64.691807 compared to gradient descent's orders-of-magnitude higher difference of 62,931.851419. This can be seen visually in Figure 7, where the simulated annealing line of best fit is closer to the analytical solution than the gradient descent line of best fit.

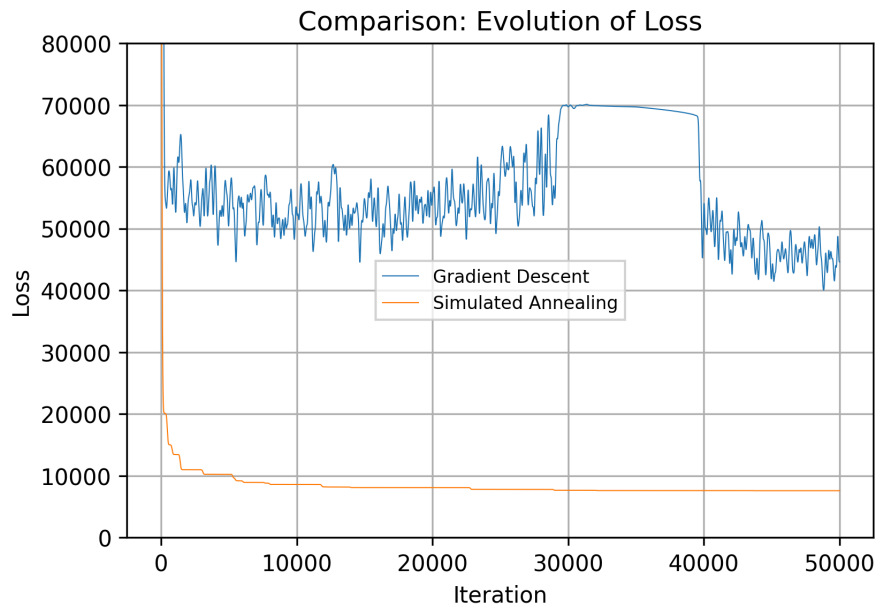


Figure 8: Comparison of loss function over iterations

Looking at the evolution of loss in both algorithms, simulated annealing was able to converge on an optimal solution much faster, reaching this by the 5000th iteration. Gradient descent, on the other hand, took 50,000 iterations to reach a solution that still led to noise in the loss function (it did not converge on a clear minimum).

7 | Conclusion

Not sure what exactly should go here. Summarizing the results would be repetitive?

- could more iterations have been better for gradient descent?
- explain why simulated annealing may have done better: Gradient descent has limitations, especially in cases of non-convex loss functions or when the data has multiple local minima. SA is better at escaping those local minima early on.
- extensions - other optimizers, combinations of methods. Levenberg-Marquardt actually combines gradient descent and Newton's method, which uses 2nd order derivatives.
- what else to put here? need a direct answer to the research question.

Bibliography

Elmetwally, T. (2023). *Car Information Dataset* [Computer software]. <https://www.kaggle.com/datasets/tawfikelmetwally/automobile-dataset>

Sun, S., Cao, Z., Zhu, H., & Zhao, J. (2019, October 23). *A Survey of Optimization Methods from a Machine Learning Perspective*. <http://arxiv.org/abs/1906.06821>